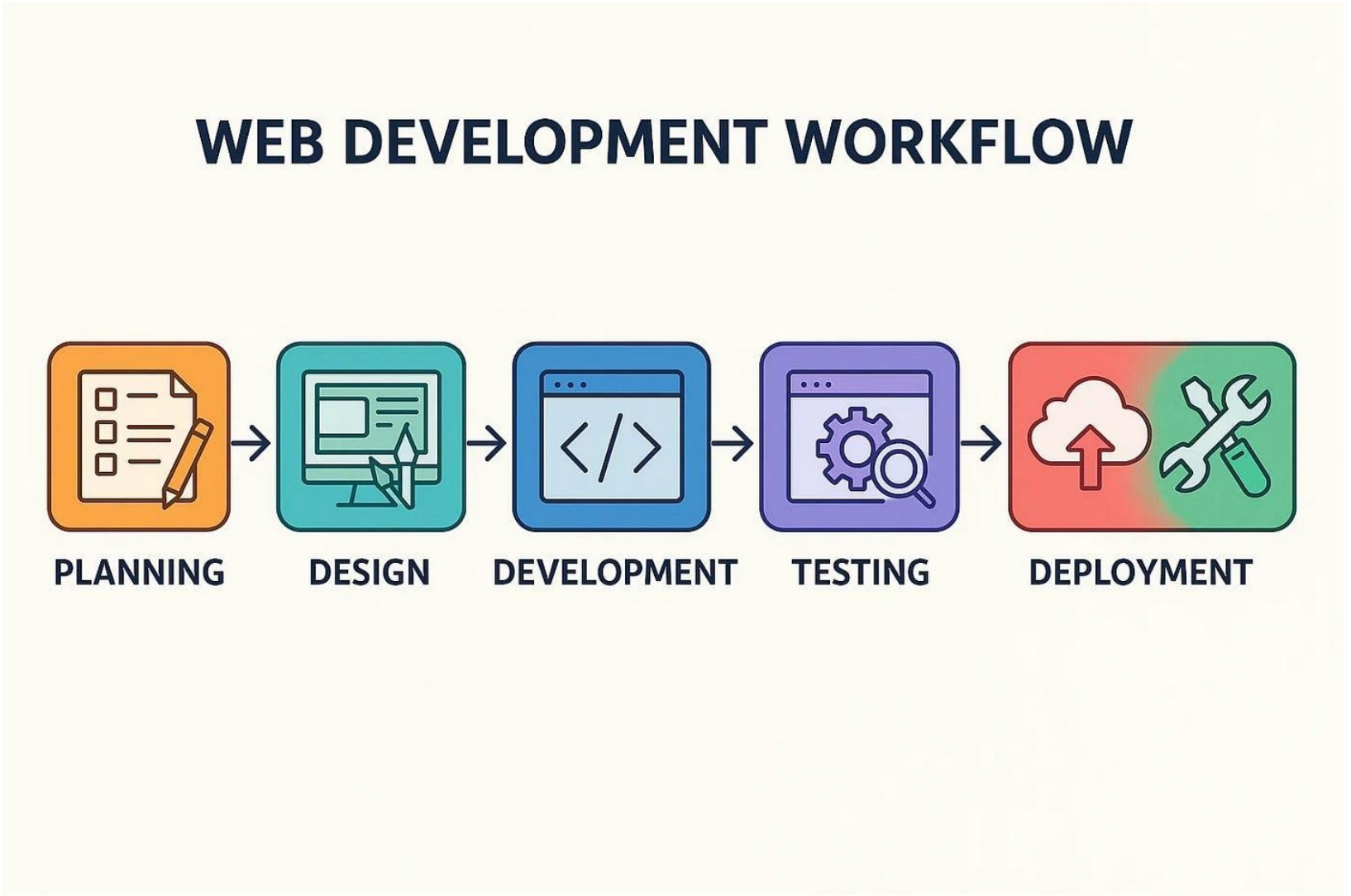
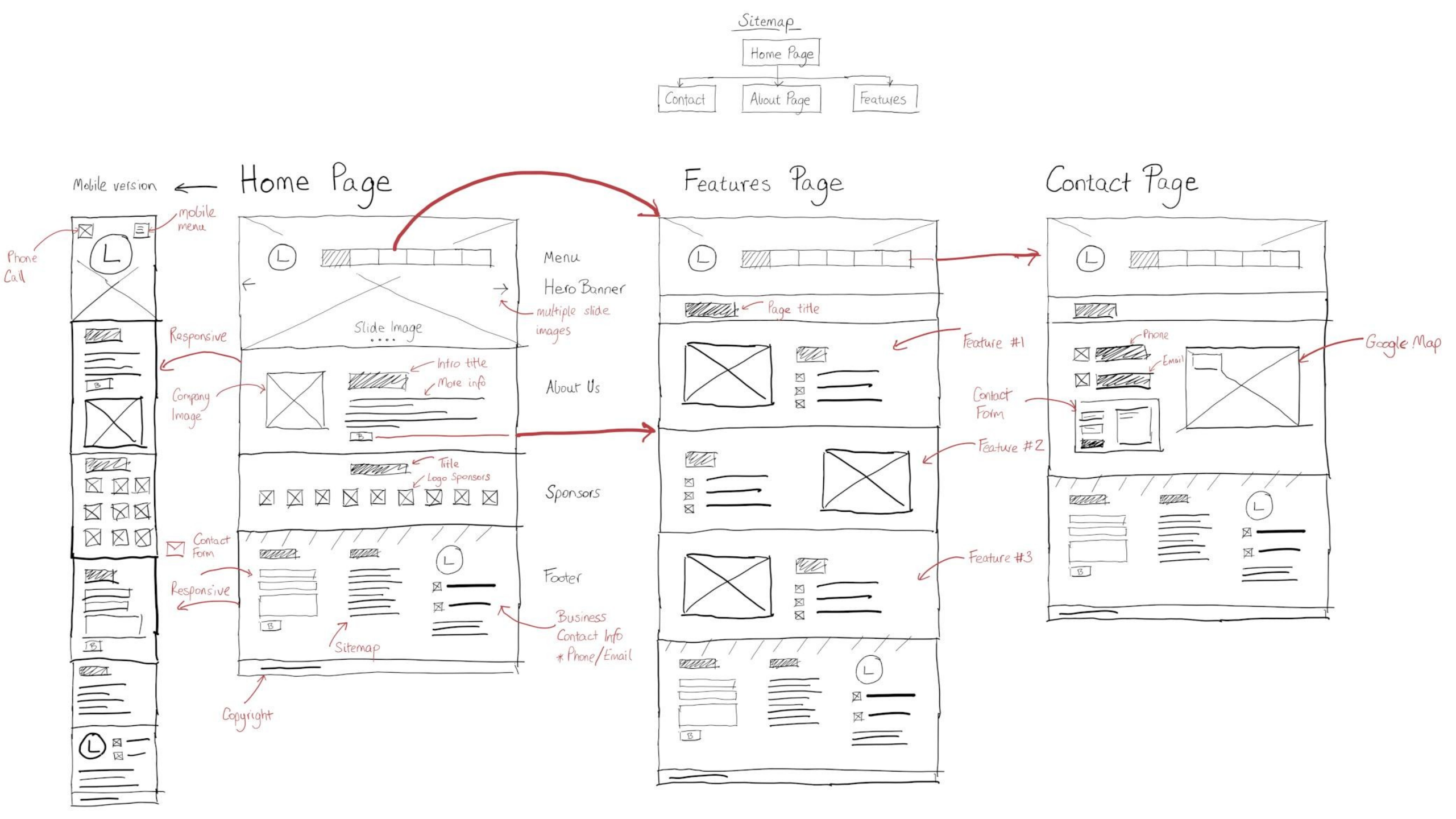
WEBSITE DEVELOPMENT

Website Development Process Phases

1. Planning – defines goals, gather requirements
2. Design – create user flows, visual mockups
3. Development – code and build website features
4. Testing – check usability, fix issues
4. Deployment – launch site to live environment
4. Monitoring and Maintenance – track performance, resolve issues
4. Continuous Improvement – update and enhance post-launch

Website Development Process Phases




Sitemap
Home Page
Contact
About Page
Features
Mobile version
Home Page
Features Page
Contact Page
mobile menu
Phone Call
Responsive
Company Image
Contact Form
Responsive
Copyright
Menu
Hero Banner
multiple slide images
Slide Image
Intro title
More info
About Us
Title
Logo Sponsors
Sponsors
Footer
Sitemap
Business Contact Info
* Phone/Email
Page title
Feature #1
Feature #2
Feature #3
Phone
Email
Google Map

WEB DEVELOPMENT PROCESS



- ✓ Plan
- ✓ Design
- ✓ Build
- ✓ Improve

1 CLIENT / IDEA

A client has a need or idea for a website. This is where the project begins.

2 PROJECT PROPOSAL

The team creates a project proposal that outlines the idea, objectives, and deliverables.

3 REQUIREMENTS DOCUMENT

Identify the functional and non-functional requirements of the website.

4 PROJECT PLAN

Plan the tasks, timeline, roles, and resources for the project.

5 SITEMAP

Organize the structure and pages of the website.

6 WIREFRAME / DESIGN

Create wireframes and a style guide for the visual design.

7 CODING STARTS

Begin building the website using HTML and CSS.

8 DEVELOPMENT LOG

Document the work done, problems encountered, and solutions.

9 TESTING

Check the website for bugs, functionality, and responsiveness.

10 BUG REPORT

List and describe the issues found, including severity and steps to reproduce.

11 CLIENT FEEDBACK

Receive feedback from the client/teacher and discuss improvements.

12 CHANGE REQUEST

Document new requests or changes from the client (e.g., add a gallery page).

13 REVISION

Implement the approved changes and fix the issues.

14 FINAL TESTING

Make sure all features work and meet the requirements.

15 DEPLOYMENT

Prepare the final version and publish the website (e.g., GitHub, hosting, or local).

16 FINAL DOCUMENTATION

Compile all project documents (overview, requirements, code notes, testing, etc.).

17 FINAL PRESENTATION

Present the completed website and project process to the client/ teacher.

18 RETROSPECTIVE

Reflect on what went well, what to improve, and lessons learned.

COMPUTER PROGRAMMING (.NET TECHNOLOGY)

Cascading Style Sheets

Lesson 1: CSS Fundamentals • Lesson 2: CSS Selectors

LESSON

1

CSS Fundamentals

Definition, role, syntax, and types of Cascading Style Sheets

What You Will Be Able To Do

Unpacked Performance Standards

1. Apply Cascading Style Sheet (CSS) fundamentals in enhancing webpage appearance and design.

2. Utilize CSS syntax, properties, and values in webpage styling.

3. Differentiate the types and functions of CSS in web development.

Origin and Development



CSS was developed by the World Wide Web Consortium (W3C), with significant contributions from **Håkon Wium Lie** and **Bert Bos**. Håkon Wium Lie first proposed the concept of CSS in **1994** to address the challenges of document presentation on the web, which was previously controlled solely by HTML.

Origin and Development

The primary goal of CSS was to separate the content of a document (HTML) from its presentation aspects (fonts, colors, layout). This separation made web development more accessible and allowed web pages to adapt more easily to different devices.

Early Version

1. CSS1

The first official specification for CSS, known as **CSS1**, was published in **December 1996**. It introduced basic features for text styling, font properties, margins, padding, color and background styling, and the box model.

Early Version

2. CSS2

Released in **May 1998**, **CSS2** expanded upon CSS1 by adding more sophisticated selectors, positioning capabilities, media types, and additional properties to enhance styling options.

Evolution and Modern CSS

1. CSS 2.1

An important revision known as CSS 2.1 addressed inconsistencies and errors found in CSS2. It became a W3C Recommendation in June 2011 and is widely supported across all major browsers.

Evolution and Modern CSS

2.CSS3

Unlike previous versions, **CSS3** was not released as a single, monolithic specification. Instead, it was divided into several separate modules, each introducing new features and capabilities. Key modules include:

- **Selectors**
- **Box Model**
- **Backgrounds and Borders**
- **Text Effects**
- **2D/3D Transformations**
- **Animations**
- **Flexbox**
- **Grid Layout**

Key Features and Innovations

- **Responsive Design:** CSS3 introduced media queries, enabling the creation of responsive designs that adapt to different screen sizes and resolutions.
- **Advanced Layouts:** Flexbox and Grid provide powerful layout mechanisms, simplifying the creation of complex web layouts that are responsive and adaptable.
- **Visual Enhancements:** Transitions, animations, and transformations allow for the creation of dynamic, interactive user interfaces without relying on JavaScript.

Key Features and Innovations

- **Responsive Design:** CSS3 introduced media queries, enabling the creation of responsive designs that adapt to different screen sizes and resolutions.
- **Advanced Layouts:** Flexbox and Grid provide powerful layout mechanisms, simplifying the creation of complex web layouts that are responsive and adaptable.
- **Visual Enhancements:** Transitions, animations, and transformations allow for the creation of dynamic, interactive user interfaces without relying on JavaScript.

LESSON 1 • DISCUSSION

What Is CSS — and Why Does It Matter?

CSS (Cascading Style Sheets) is the language used to describe how an HTML document should look on screen.

1 Definition

A stylesheet language that controls the presentation of HTML elements — colors, fonts, spacing, and layout.

2 Role in Web Development

Separates content (HTML) from design (CSS), letting one stylesheet control the look of many pages consistently.

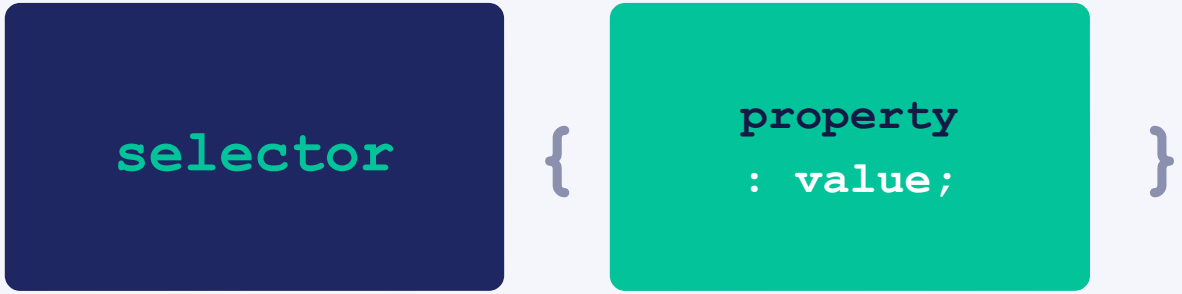
3 Why It's Powerful

One change in a CSS file can instantly update the appearance of an entire website — no need to edit every page.

4 Cascading Behavior

Styles "cascade" — rules can come from multiple sources, and specificity/order decide which one applies.

The CSS Syntax Structure




```
p {
  color: navy;
  font-size: 16px;
}
```

Selector

What element to style

Declaration

The property to change and its value

Rule of thumb: every CSS declaration ends with a semicolon (;) and every rule is wrapped in curly braces { }. A selector can hold many declarations.

The CSS Syntax Structure

```
h1 {
    color: green;
}
```

Three Ways to Apply CSS

1

Inline CSS

Written directly inside an HTML tag using the style attribute. Best for one-off, single-element tweaks.

Least reusable

2

Internal (Embedded) CSS

Placed inside a <style> tag within the <head> of a single HTML page. Good for page-specific styling.

One page only

3

External CSS

Written in a separate .css file and linked with <link>. Powers consistent styling across an entire website.

Best practice

Same Page, Three Ways to Style It

Here is one paragraph styled using inline, internal, and external CSS. Notice how the external file (right) can be reused by many pages.

```
<!-- 1. Inline -->
<p style="color: navy;">Hello!</p>

<!-- 2. Internal (in <head>) -->
<style>
  p { color: navy; }
</style>

<!-- 3. External (style.css, linked) -->
<link rel="stylesheet" href="style.css">
```

LESSON 1 • ASSESSMENT

Quick Check: CSS Fundamentals

1. What does the acronym CSS stand for?

2. Write the general syntax structure of a CSS rule (selector, property, value).

3. Name the three ways CSS can be applied to an HTML document.

4. Which type of CSS is recommended for styling an entire multi-page website consistently, and why?

5. True or False: Internal CSS is written inside a separate .css file.

LESSON 1 • ACTIVITY

Hands-On: Style It Your Way

Activity: CSS Styling Warm-Up

Apply what you learned by styling a plain HTML page using an external CSS file.

1. Open the provided plain.html file (an unstyled student profile page).
2. Create a new style.css file and link it using <link rel="stylesheet">.
3. Apply at least 3 CSS properties (e.g., color, font-family, background-color).
4. Preview your page in a browser and compare it with the original.

LESSON

2

CSS Selectors

Targeting the right elements with class and ID selectors

CPNT 1.11

What You Will Be Able To Do

1. Utilize different CSS selectors to customize and style webpage elements effectively.
2. Apply class and ID selectors appropriately in webpage styling.
3. Design visually enhanced webpages using various CSS selectors.

LESSON 2 • DISCUSSION

What Are CSS Selectors?

A selector is the part of a CSS rule that tells the browser which HTML element(s) to style.

.

Class Selector

Written with a dot (.classname). Can be reused on many elements — ideal for styling a repeated group, like all buttons.

#

ID Selector

Written with a hash (#idname). Applies to only one unique element per page — ideal for a one-of-a-kind section like the header.

↻

Reusability

A class can be applied to unlimited elements; an ID should be used only once per page (must stay unique).

⚖

Specificity

When rules conflict, an ID selector overrides a class selector — IDs carry more "weight" in the cascade.

Class vs. ID — Side by Side

Class Selector

```
<p class="highlight">Text</p>

.highlight {
  color: teal;
}
```

- Reusable on many elements
- Selector starts with a dot (.)
- Great for repeated styles (cards, buttons)

ID Selector

```
<h1 id="main-title">Text</h1>

#main-title {
  text-align: center;
}
```

- Unique — one element only
- Selector starts with a hash (#)
- Great for one-of-a-kind sections

Styling a Mini Webpage with Selectors

This snippet styles two reusable "card" elements with a class, and one unique banner with an ID.

```
<!-- HTML -->
<div id="banner">Welcome!</div>
<div class="card">Skill 1</div>
<div class="card">Skill 2</div>

/* CSS */
#banner { background: #1E2761; }
.card { border: 1px solid #02C39A; }
```

LESSON 2 • ASSESSMENT

Quick Check: CSS Selectors

1. Which symbol is used to write a class selector?
2. Which symbol is used to write an ID selector?
3. Can the same class be applied to more than one element on a page?
4. How many elements on a single page should share the same ID?
5. Write the CSS rule that selects an element with the class "menu".

Thank you!